

**МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ДЕРЖАВНИЙ ВИЩИЙ НАВЧАЛЬНИЙ ЗАКЛАД
«УЖГОРОДСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ»
ІНЖЕНЕРНО-ТЕХНІЧНИЙ ФАКУЛЬТЕТ
КАФЕДРА КОМП'ЮТЕРНИХ СИСТЕМ ТА МЕРЕЖ**

ЗВІТ

МІНІ-ПРОЄКТ

з дисципліни «Програмування мікроконтролерів»

Студента 2-го курсу спеціальності

123 – «Комп'ютерна інженерія»

Євтушика Івана Івановича

м. Ужгород – 2026 р.

ЗМІСТ

1. Стан очікування введення пароля <i>ENTER</i>	2
2. Введення пароля з матричної клавіатури.....	3
3. Повідомлення <i>TRUE</i> після правильного пароля.....	4
4. Зворотний відлік 60 секунд.....	5
5. Повідомлення <i>FALSE</i> після неправильного пароля.....	6
6. Повернення системи у стан <i>ENTER</i>	7
ВИСНОВКИ.....	8
Додаток А. Лістинг програми.....	9

МІНІ-ПРОЄКТ «Сейф»

У результаті виконання мініпроєкту було реалізовано програму електронного сейфа на базі *PSoC*. Користувач вводить пароль за допомогою матричної клавіатури, після чого натискає кнопку # для підтвердження введення. Програма порівнює введений пароль із заданим у коді. Якщо пароль правильний, на семисегментному індикаторі з'являється повідомлення *TRUE*, після чого запускається зворотний відлік 60 секунд. Якщо пароль неправильний, на індикаторі з'являється повідомлення *FALSE*, після чого система повертається у режим очікування введення пароля.

1. Стан очікування введення пароля *ENTER*

Після запуску програми система переходить у початковий стан очікування. На семисегментному індикаторі відображається напис *ENTER*. Це означає, що пристрій готовий до введення пароля з матричної клавіатури.

У цьому стані програма не виконує перевірку пароля, а лише очікує натискання цифрових клавіш. Після натискання першої цифри дисплей очищується, і введені цифри починають відображатися на індикаторі.

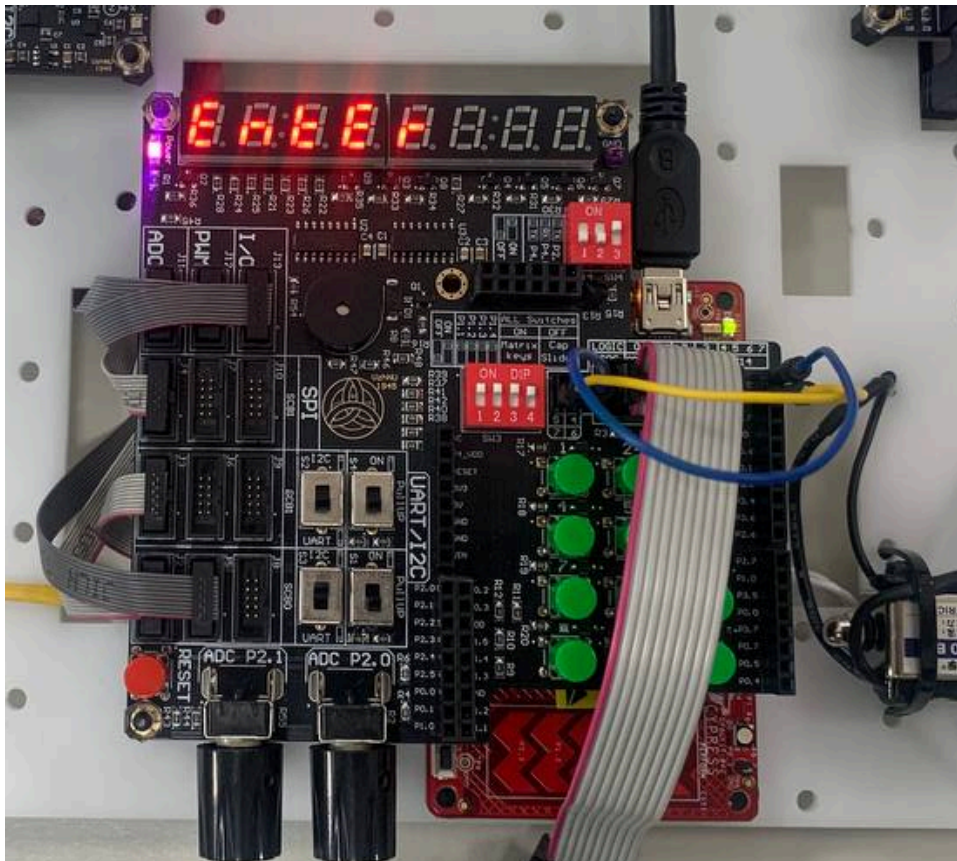


Рисунок 1 — Стан очікування введення пароля *ENTER*

2. Введення пароля з матричної клавіатури

Після натискання цифрових клавiш користувач вводить пароль. У програмі правильний пароль задано як 1234.

Кожна натиснута цифра записується у масив `input[]` і одночасно виводиться на семисегментний індикатор. Це дозволяє користувачу бачити процес введення пароля. Коли введено чотири цифри, користувач натискає кнопку #, після чого програма переходить до перевірки пароля.

Якщо потрібно скинути введення, використовується кнопка *. Після її натискання введені дані очищуються, а на дисплеї знову з'являється напис *ENTER*.

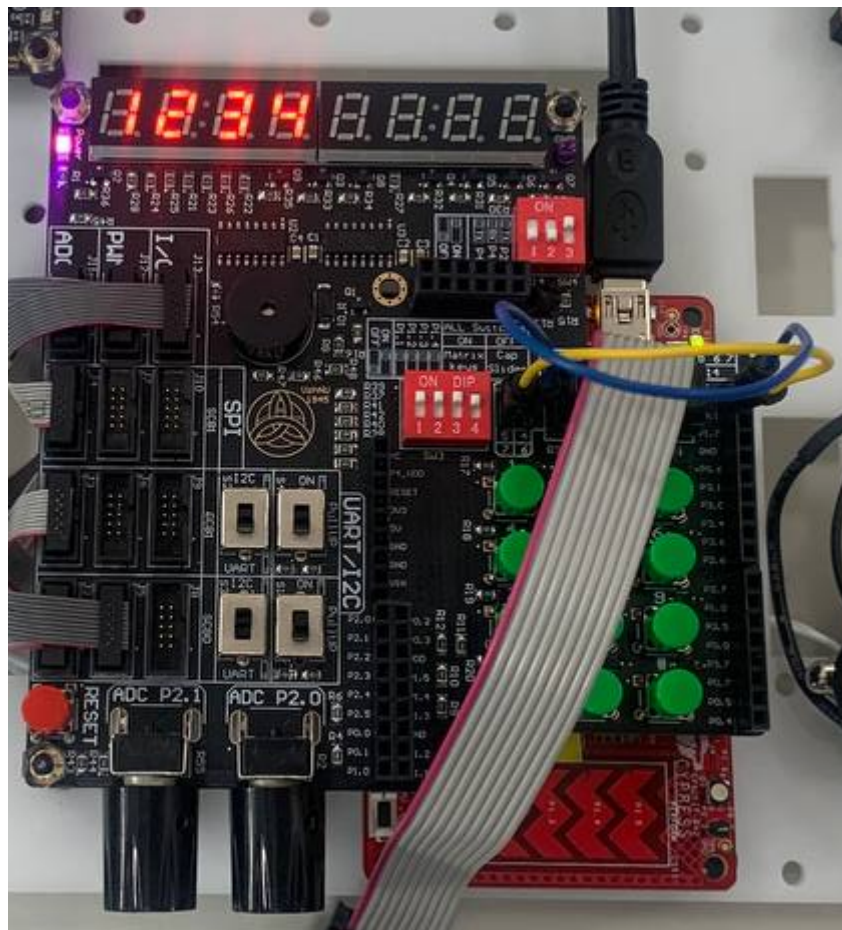


Рисунок 2 — Введення пароля з матричної клавіатури

3. Повідомлення *TRUE* після правильного пароля

Якщо користувач ввів правильний пароль 1234 і натиснув кнопку #, програма виконує перевірку за допомогою функції *checkPassword()*.

У випадку правильного введення на семисегментному індикаторі з'являється напис *TRUE*. Це повідомлення підтверджує, що пароль введено правильно і доступ до сейфа дозволено.

Повідомлення *TRUE* відображається протягом 10 секунд. Після завершення цього часу програма автоматично переходить до режиму зворотного відліку.

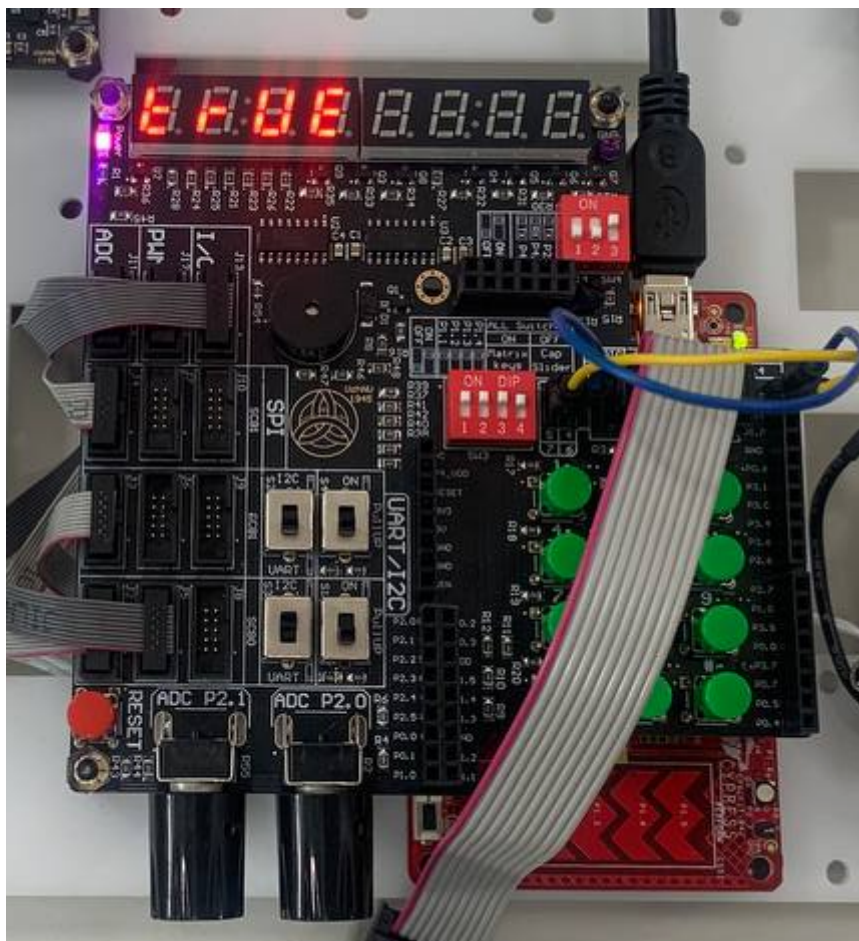


Рисунок 3 — Повідомлення *TRUE* після правильного введення пароля

4. Зворотний відлік 60 секунд

Після відображення повідомлення *TRUE* запускається таймер на 60 секунд. На перших чотирьох семисегментних індикаторах відображається час у форматі хвилини та секунди.

Початкове значення таймера дорівнює 60 секундам. У програмі це задається змінною: `static uint16_t seconds = 60;`

Функція *updateCountdownDisplay()* перетворює кількість секунд у формат часу та виводить його на індикатор. Під час роботи таймера значення часу зменшується кожну секунду. Для формування інтервалу в одну секунду використовується таймер і прапорець *second_flag*.

Цей етап показує, що після правильного введення пароля система переходить у робочий режим і виконує задану дію – зворотний відлік.

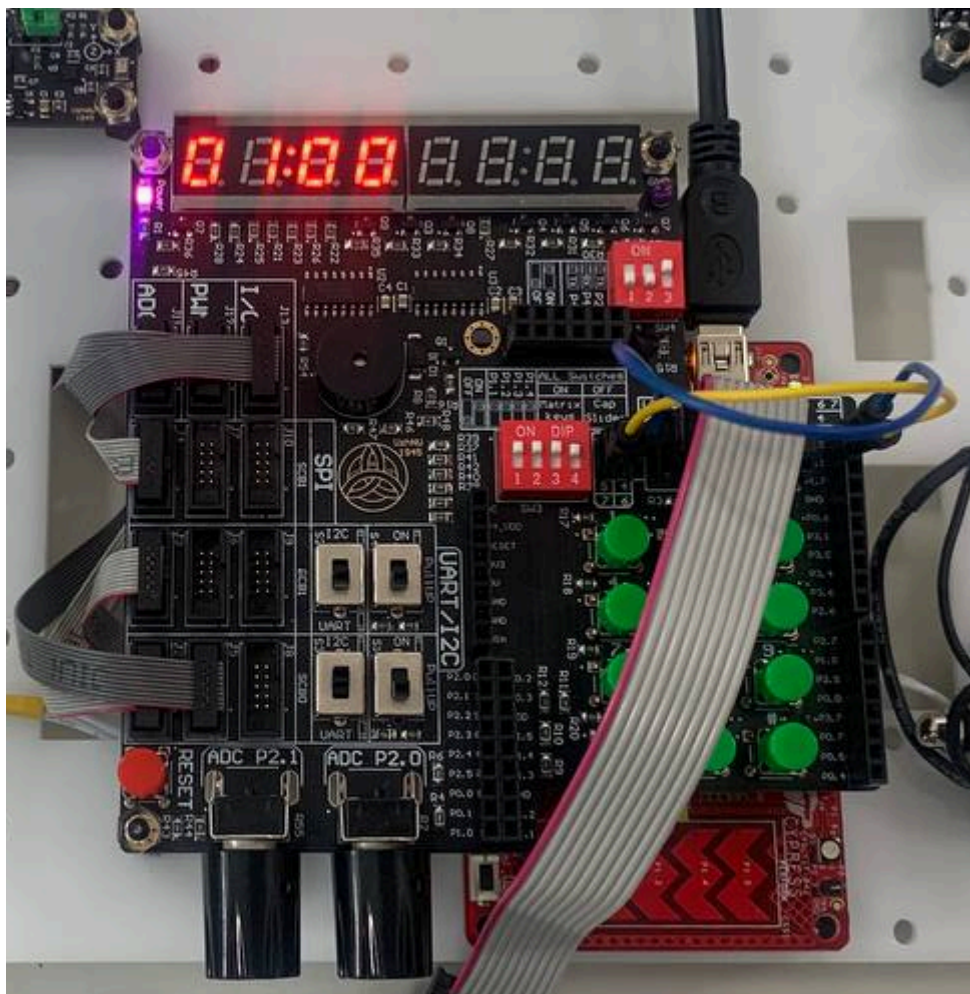


Рисунок 4 — Зворотний відлік 60 секунд після правильного пароля

5. Повідомлення *FALSE* після неправильного пароля

Якщо користувач вводить неправильний пароль і натискає кнопку #, програма також виконує перевірку функцією *checkPassword()*. Якщо хоча б одна цифра не збігається із заданим паролем, результат перевірки є неправильним.

У такому випадку на семисегментному індикаторі з'являється напис *FALSE*. Це повідомлення інформує користувача, що пароль введено неправильно і доступ до сейфа заборонено.

Повідомлення *FALSE* відображається протягом 3 секунд. Після цього введені дані очищуються, і система повертається у початковий стан очікування.

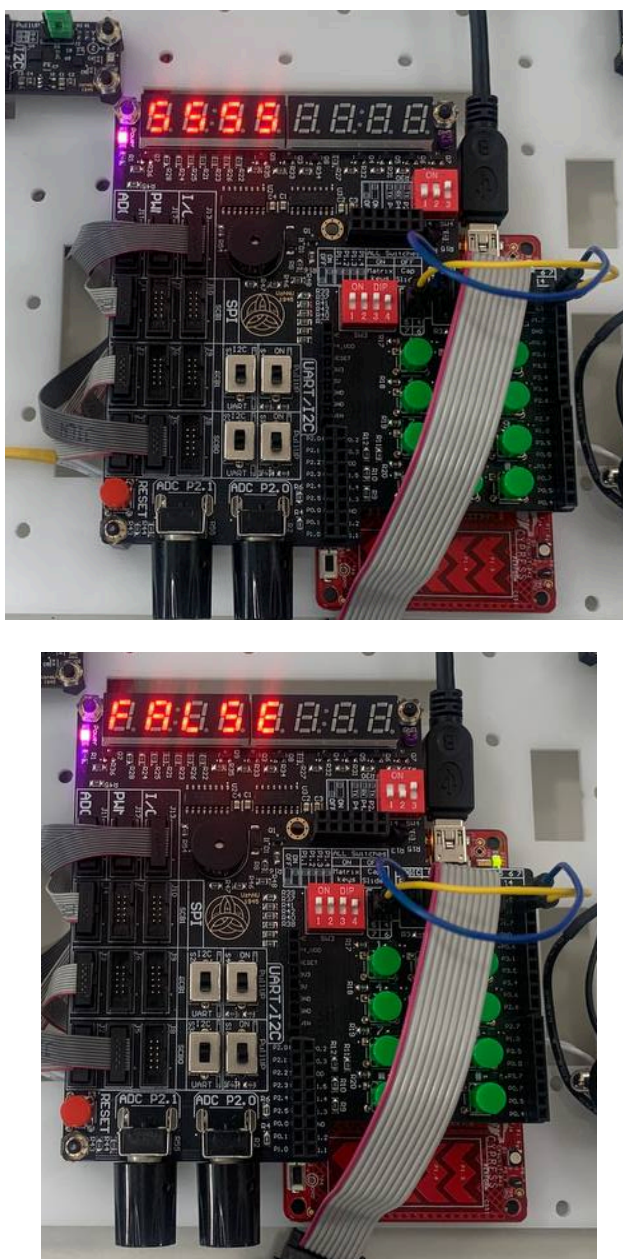


Рисунок 5 — Повідомлення *FALSE* після неправильного введення пароля

6. Повернення системи у стан *ENTER*

Після неправильного введення пароля або після натискання кнопки * система очищує введені дані та повертається до початкового стану. На семисегментному індикаторі знову з'являється напис *ENTER*.

Це означає, що програма готова до нового введення пароля. Такий режим роботи дозволяє багаторазово вводити пароль без перезавпуску мікроконтролера.

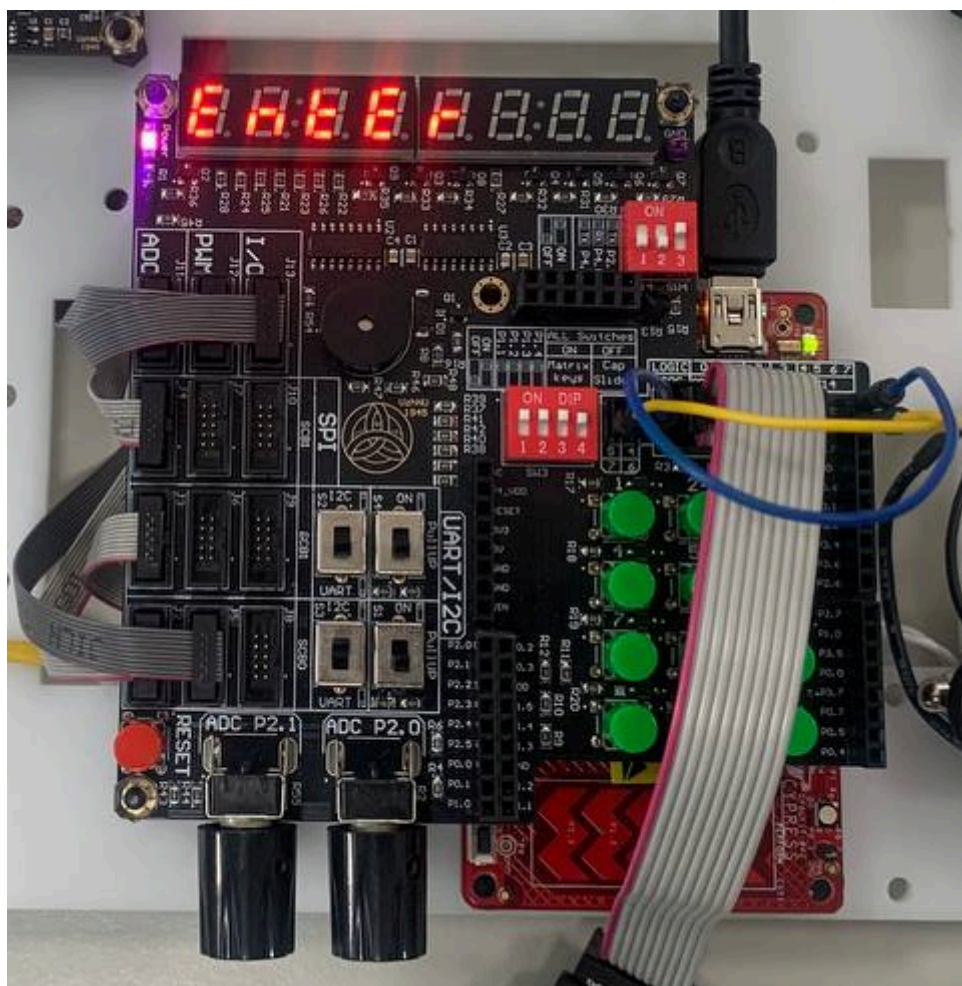


Рисунок 6 — Повернення системи у режим очікування *ENTER*

ВИСНОВКИ

У ході виконання мініпроєкту «Сейф» було реалізовано систему введення та перевірки пароля на базі *PSoC* з використанням матричної клавіатури та семисегментного індикатора. Програма працює у кількох станах: очікування введення пароля, введення цифр, перевірка пароля, виведення повідомлення *TRUE* або *FALSE*, а також запуск зворотного відліку часу.

Було реалізовано виведення напису *ENTER*, який показує, що система готова до введення пароля. Після введення чотирьох цифр і натискання кнопки *#* програма порівнює введений пароль із заданим у коді. Якщо пароль правильний, на індикаторі відображається повідомлення *TRUE*, після чого запускається відлік 60 секунд. Якщо пароль неправильний, виводиться повідомлення *FALSE*, а система повертається у режим очікування.

У результаті роботи було закріплено навички роботи з динамічною індикацією, таймером, перериваннями, матричною клавіатурою та зсувними регістрами *74HC595*. Надані зображення підтверджують правильність виконання всіх основних етапів програми: очікування введення, введення пароля, правильну та неправильну перевірку, а також роботу таймера зворотного відліку. Отже, мініпроєкт виконано успішно.

Додаток А. Лістинг програми

```
#include "project.h"

#define SEG_0      0xC0
#define SEG_1      0xF9
#define SEG_2      0xA4
#define SEG_3      0xB0
#define SEG_4      0x99
#define SEG_5      0x92
#define SEG_6      0x82
#define SEG_7      0xF8
#define SEG_8      0x80
#define SEG_9      0x90

#define SEG_A      0x88
#define SEG_E      0x86
#define SEG_F      0x8E
#define SEG_L      0xC7
#define SEG_S      0x92
#define SEG_U      0xC1
#define SEG_T      0x87
#define SEG_R      0xAF
#define SEG_N      0xAB
#define SEG_BLANK  0xFF

static const uint8_t digit_code[10] = {
    SEG_0, SEG_1, SEG_2, SEG_3, SEG_4,
    SEG_5, SEG_6, SEG_7, SEG_8, SEG_9
};

static uint8_t display_data[8] = {
    SEG_BLANK, SEG_BLANK, SEG_BLANK, SEG_BLANK,
    SEG_BLANK, SEG_BLANK, SEG_BLANK, SEG_BLANK
};

static uint8_t display_dot[8] = {
    0, 0, 0, 0, 0, 0, 0, 0
};

static uint8_t password[4] = {1, 2, 3, 4};
static uint8_t input[4] = {0, 0, 0, 0};
static uint8_t input_counter = 0;

static uint8_t key_map[4][3] = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9},
    {16, 0, 17} // 16 = *, 17 = #
};

void (*write_col[3])(uint8) = {
    COLUMN_0_Write,
    COLUMN_1_Write,
    COLUMN_2_Write
};

void (*set_col_dm[3])(uint8) = {
    COLUMN_0_SetDriveMode,
    COLUMN_1_SetDriveMode,
    COLUMN_2_SetDriveMode
};

uint8 (*read_row[4])(void) = {
```

```

    ROW_0_Read,
    ROW_1_Read,
    ROW_2_Read,
    ROW_3_Read
};

#define STATE_WAIT          0
#define STATE_INPUT        1
#define STATE_TRUE_MESSAGE 2
#define STATE_FALSE_MESSAGE 3
#define STATE_COUNTDOWN    4

static volatile uint8_t state = STATE_WAIT;

static volatile uint8_t led_counter = 0;
static volatile uint32_t state_ms = 0;
static volatile uint16_t countdown_ms = 0;
static volatile uint8_t second_flag = 0;

static uint16_t seconds = 60;

static void FourDigit74HC595_sendData(uint8_t data)
{
    for (uint8_t i = 0; i < 8; i++)
    {
        Pin_DO_Write((data & (0x80 >> i)) ? 1 : 0);
        Pin_CLK_Write(1);
        Pin_CLK_Write(0);
    }
}

static void FourDigit74HC595_sendOneCode(uint8_t position, uint8_t segment_code,
uint8_t dot)
{
    if (position >= 8)
    {
        FourDigit74HC595_sendData(0xFF);
        FourDigit74HC595_sendData(0xFF);
        Pin_Latch_Write(1);
        Pin_Latch_Write(0);
        return;
    }

    FourDigit74HC595_sendData(0xFF & ~(1 << position));

    if (dot)
    {
        FourDigit74HC595_sendData(segment_code & 0x7F);
    }
    else
    {
        FourDigit74HC595_sendData(segment_code);
    }

    Pin_Latch_Write(1);
    Pin_Latch_Write(0);
}

static void clearDisplay(void)
{
    for (uint8_t i = 0; i < 8; i++)
    {
        display_data[i] = SEG_BLANK;
        display_dot[i] = 0;
    }
}

```

```

    }
}

static void showENTER(void)
{
    clearDisplay();

    display_data[0] = SEG_E;
    display_data[1] = SEG_N;
    display_data[2] = SEG_T;
    display_data[3] = SEG_E;
    display_data[4] = SEG_R;
}

static void showTRUE(void)
{
    clearDisplay();

    display_data[0] = SEG_T;
    display_data[1] = SEG_R;
    display_data[2] = SEG_U;
    display_data[3] = SEG_E;
}

static void showFALSE(void)
{
    clearDisplay();

    display_data[0] = SEG_F;
    display_data[1] = SEG_A;
    display_data[2] = SEG_L;
    display_data[3] = SEG_S;
    display_data[4] = SEG_E;
}

static void updateCountdownDisplay(void)
{
    uint8_t minutes = seconds / 60;
    uint8_t sec = seconds % 60;

    clearDisplay();

    display_data[0] = digit_code[minutes / 10];
    display_data[1] = digit_code[minutes % 10];
    display_data[2] = digit_code[sec / 10];
    display_data[3] = digit_code[sec % 10];

    // крапка після хвилин як розділювач
    display_dot[1] = 1;
}

static void setState(uint8_t new_state)
{
    CyGlobalIntDisable;

    state = new_state;
    state_ms = 0;
    countdown_ms = 0;
    second_flag = 0;

    CyGlobalIntEnable;
}

static void resetInput(void)

```



```

{
    input_counter = 0;

    for (uint8_t i = 0; i < 4; i++)
    {
        input[i] = 0;
    }
}

static uint8_t checkPassword(void)
{
    for (uint8_t i = 0; i < 4; i++)
    {
        if (input[i] != password[i])
        {
            return 0;
        }
    }

    return 1;
}

static uint8_t getKey(void)
{
    for (uint8_t c = 0; c < 3; c++)
    {
        for (uint8_t i = 0; i < 3; i++)
        {
            set_col_dm[i] (CY_SYS_PINS_DM_DIG_HIZ);
        }

        set_col_dm[c] (CY_SYS_PINS_DM_STRONG);
        write_col[c] (0);

        for (uint8_t r = 0; r < 4; r++)
        {
            if (read_row[r]() == 0)
            {
                CyDelay(20);

                while (read_row[r]() == 0)
                {
                }

                return key_map[r][c];
            }
        }
    }

    return 255;
}

CY_ISR(Timer_Int_Handler2)
{
    Timer_1_ReadStatusRegister();

    FourDigit74HC595_sendOneCode(
        led_counter,
        display_data[led_counter],
        display_dot[led_counter]
    );

    led_counter++;
}

```

```

    if (led_counter > 7)
    {
        led_counter = 0;
    }

    state_ms++;

    if (state == STATE_COUNTDOWN)
    {
        countdown_ms++;

        if (countdown_ms >= 1000)
        {
            countdown_ms = 0;
            second_flag = 1;
        }
    }
}

int main(void)
{
    CyGlobalIntEnable;

    showENTER();

    Timer_Int_StartEx(Timer_Int_Handler2);
    Timer_1_Start();

    for (;;)
    {
        uint8_t key = getKey();

        if (state == STATE_WAIT || state == STATE_INPUT)
        {
            if (key != 255)
            {
                // Натиснута цифра
                if (key <= 9)
                {
                    if (state == STATE_WAIT)
                    {
                        clearDisplay();
                        resetInput();
                        setState(STATE_INPUT);
                    }

                    if (input_counter < 4)
                    {
                        input[input_counter] = key;
                        display_data[input_counter] = digit_code[key];
                        input_counter++;
                    }
                }

                // Натиснуто #
                if (key == 17)
                {
                    if (input_counter == 4)
                    {
                        if (checkPassword())
                        {
                            showTRUE();
                            setState(STATE_TRUE_MESSAGE);
                        }
                    }
                }
            }
        }
    }
}

```

```

        else
        {
            showFALSE();
            setState(STATE_FALSE_MESSAGE);
        }
    }

    // Натиснуто *
    if (key == 16)
    {
        resetInput();
        showENTER();
        setState(STATE_WAIT);
    }
}

if (state == STATE_TRUE_MESSAGE)
{
    if (state_ms >= 10000)
    {
        seconds = 60;
        updateCountdownDisplay();
        setState(STATE_COUNTDOWN);
    }
}

if (state == STATE_FALSE_MESSAGE)
{
    if (state_ms >= 3000)
    {
        resetInput();
        showENTER();
        setState(STATE_WAIT);
    }
}

if (state == STATE_COUNTDOWN)
{
    if (second_flag)
    {
        second_flag = 0;

        if (seconds > 0)
        {
            seconds--;
            updateCountdownDisplay();
        }
    }

    if (key == 16)
    {
        resetInput();
        showENTER();
        setState(STATE_WAIT);
    }
}
}
}

```